Can LLM Agents Test Safety-Critical Code? An ISO 26262 Baseline Study

Anonymous[1]

[1]*Anonymous Institution, Anonymous Country*

Abstract

Can LLM agents test safety-critical code? We study this question through an ISO 26262 baseline focused on unit testing and structural coverage requirements.

This short paper presents a controlled baseline analysis of GPT-4 Turbo, configured as a **multi-agent system** with dedicated test agents for selected Automotive Safety Integrity Levels (ASIL A, B, and D). Each agent was specialised to meet the respective ISO 26262-6 Clause 9 verification objectives, from statement and branch coverage to full MC/DC independence. At the time of writing, GPT-4 Turbo provides a *128k-token context window* and reduced cost; we *fix the model snapshot and decoding parameters* (temperature, top-p, seed) to control variance [1, 2]. Using a controlled test harness with GoogleTest and `gcov`, fixed toolchain versions, no-autofix governance policies, and coverage-guided re-prompting, we assess performance under **Automotive SPICE SWE.4** software unit verification.

Across **32 experiments** (agent–unit tasks), agents achieve high line/branch coverage for ASIL A/B, but fail to establish MC/DC independence in the ASIL D experiment; boundary-value gaps persist. We contribute coverage and autonomy baselines, a failure-pattern taxonomy, and a transparent evaluation protocol. The study clarifies where multi-agent LLM setups already add value for SWE.4 activities and where human oversight and improved methods (e.g., MC/DC witness search) remain necessary.

Keywords

Large Language Models, Multi-Agent Systems, GPT-4 Turbo, ISO 26262, Automotive SPICE, SWE.4, ASIL, MC/DC Coverage, Unit Testing, Safety-Critical Software, Failure Pattern Taxonomy, Reproducibility

1. Introduction and Motivation

The safe and reliable operation of automotive electronic control units (ECUs) is governed by strict standards such as ISO 26262 [3] and Automotive SPICE (ASPICE) [4]. Within ASPICE, **SWE.4 = Software Unit Verification** specifies unit-level verification, including the execution of norm-compliant tests, objective result recording, and traceability from requirements to test outcomes.

Large Language Models (LLMs) have demonstrated strong code understanding, reasoning, and test generation capabilities, raising the question of whether such models could act as *test agents* in safety-critical domains. In this work, we explore a **multi-agent setup**, where each Automotive Safety Integrity Level (ASIL) is served by a dedicated LLM-based test agent. This allows specialisation toward ASIL-dependent goals (e.g., line/branch coverage vs. MC/DC independence).

Our contribution is the first controlled GPT-4 Turbo baseline for SWE.4-compliant unit testing with ISO 26262-6 evidence. We report structural coverage, autonomy metrics, and failure patterns, and discuss reproducibility challenges. The work addresses LLAIS topics on software automation, reproducibility, and AI-agent orchestration.

2. Background: ASPICE SWE.4 and ISO 26262-6

ASPICE [4] provides the industry reference for software process assessment. SWE.4 requires the selection of verification measures, execution, result recording, and bidirectional traceability of unit ↔ measure ↔ result.

LLAIS 2025: Workshop on LLM-based Agents for Intelligent Systems, August 15, 2025

© 2025 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

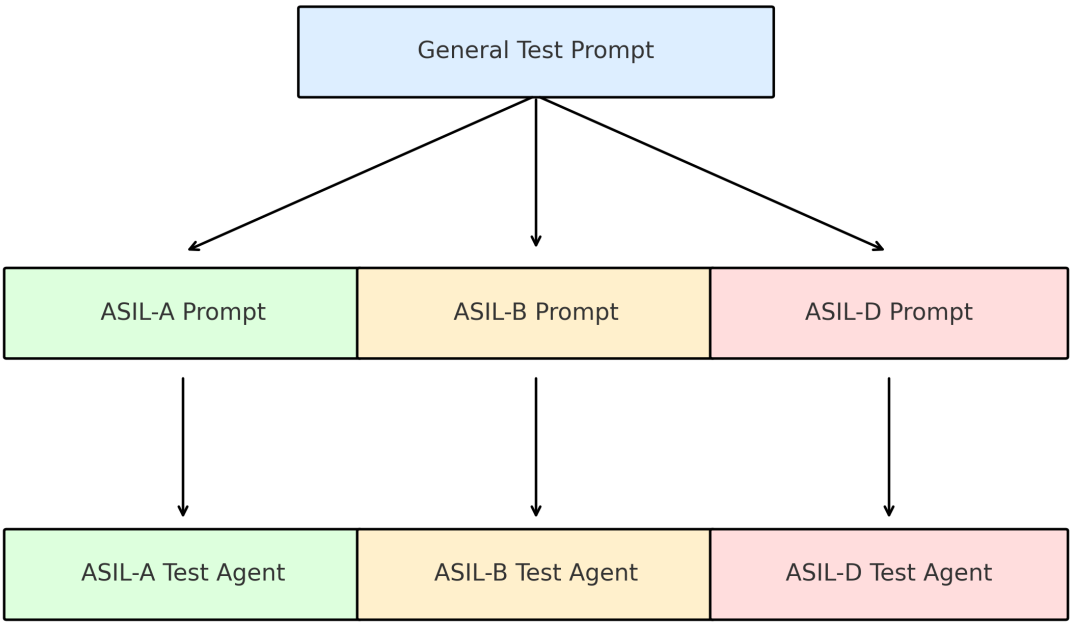

Figure 1: Multi-agent setup for ISO 26262 unit test generation: a shared general test prompt branches into ASIL-specific prompts, each linked to a dedicated LLM test agent.

ISO 26262-6 [3] specifies unit-level verification, including ASIL-dependent coverage targets (statement, branch, MC/DC). MC/DC requires that each condition in a decision can independently affect the decision outcome; we adopt the *unique-cause* interpretation for independence [5, 6].

This study investigates whether a multi-agent LLM setup can perform SWE.4 tasks and produce ISO 26262-6 evidence.

3. Method

3.1. Multi-Agent Architecture per ASIL Level

The evaluation was conducted using a multi-agent setup in which the **ASIL level (A,B,[C],D)** was assigned a *dedicated LLM-based test agent*. This design choice reflects the fact that ISO 26262 imposes distinct verification goals for each ASIL level, making a single generic prompt suboptimal.

The multi-agent setup is illustrated in Figure 1. A **General Test Prompt** specified governance policies (fixed toolchain and model version, no automatic code modifications, reproducibility requirements), toolchain integration (GoogleTest for execution, `gcov` for coverage analysis), and a uniform output format (test case tables, GoogleTest-compatible code, and HTML documentation).

From this general prompt, three **ASIL-specific prompts** were derived, each tailored to the verification objectives defined in ISO 26262-6 Clause 9:

- **ASIL A Agent**: requirement-based tests targeting 100% line coverage.
- **ASIL B Agent**: requirement-based tests extended with equivalence partitioning and boundary-value analysis, targeting 100% line and branch coverage.
- **ASIL D Agent**: ASIL B capabilities plus MC/DC independence requirement; fallback to error guessing when independence could not be achieved.

3.2. Function Set and Laboratory Setup

The agents were evaluated in a controlled laboratory environment using norm-based test tasks, partly replicating real functions from the automotive domain:

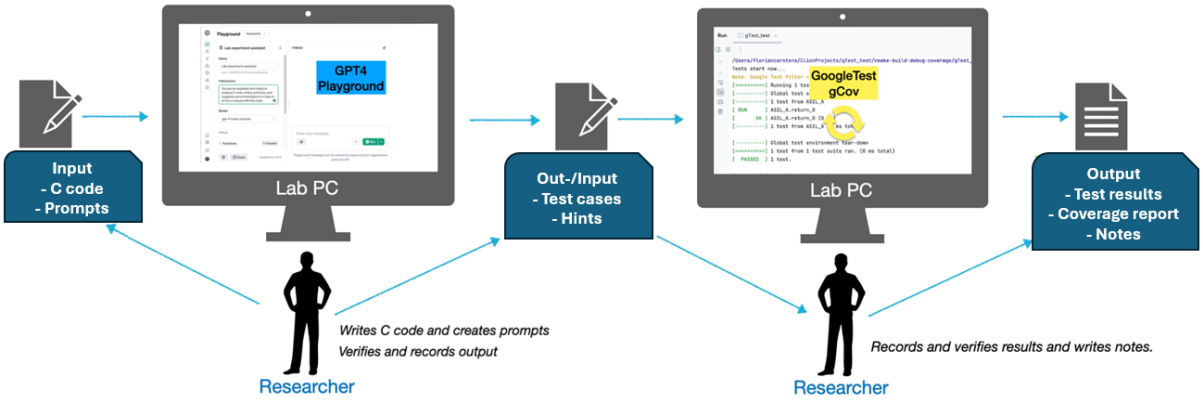

Figure 2: Laboratory setup and toolchain for SWE.4-compliant unit testing.

- **Boolean algebra functions** (`boolean_algebra.c/.h`)
- **Mathematical routines** (`math_lib.c/.h`)
- **Simplified real-world safety-relevant examples** (`real_world.c/.h`), including a NASA MC/DC case [5] and an automotive MCU state machine.

The toolchain (Figure 2) used GoogleTest for test execution, `gcov` for coverage analysis, fixed toolchain and model versions, and governance policies such as no-autofix, coverage-guided re-prompting, and full audit trails. This ensured controllability and allowed detailed mapping of results to ASPICE SWE.4 and ISO 26262-6 evidence requirements.

3.3. Measurement Definitions

Unless stated otherwise, we report *micro-averages over experiments (agent–unit tasks)*; see Appendix A for a comparison of macro vs. micro averaging.
First-Pass Success (FPS): fraction of experiments for which the very first test generation achieved the ASIL-specific coverage goal without re-prompting.
Avg. Iterations: mean number of re-prompting cycles per experiment until the coverage goal was met or a loop budget of ≤ 3 was reached.
Coverage: line/branch via `gcov`; MC/DC per ISO 26262-6 Clause 9.
Decoding/control: fixed model snapshot; `temperature=0.2`, `top_p=0.95`, `seed=42`.

3.4. Dataset

We analyse **32 agent–unit tasks** across three function families (18 boolean algebra, 12 math, 2 real-world). Agents did *not* process the same set: ASIL A attempted 23 tasks, ASIL B 8, and ASIL D 1 (MC/DC-intensive). Unless noted otherwise, all reported results are *micro-averages over experiments*; agent-level tables aggregate over the experiments each agent attempted.

3.5. MC/DC Check

For each decision with n conditions we build a decision/condition matrix and require, for every condition c_i, two test vectors x, y that differ only in c_i and flip the decision outcome (*unique-cause MC/DC*). We generate candidates from the agent-produced set and greedily reduce to an independence witness; failure is reported if no witness exists within the loop budget [5, 6].

Table 1
Coverage per agent (micro-averages over the experiments each agent attempted).

ASIL	Tasks (n)	Line (%)	Branch (%)	MC/DC (met)	Boundary Cases
A	23	100	92	N/A	Partial
B	8	100	95	N/A	Partial
D	1	98	90	No	Partial

Table 2
Failure patterns per ASIL-specific test agent (micro-averaged over experiments).

ASIL	Agent–Unit Tasks	Total Errors	Success (%)	FP	FN	INC	UAA	HIN	COM	MIS
A	23	3	87.0	2	0	1	0	0	0	0
B	8	1	87.5	0	0	0	0	0	0	1
D	1	1	0.0	0	0	0	0	0	1	0

3.6. Failure Taxonomy

FP: test fails although the reference oracle passes.
FN: test passes although a fault is injected (when applicable).
INC: contradictory outputs across runs or artifacts.
UAA: unannounced action (code modifications despite policy).
HIN: high interaction need (human confirmation required due to toolchain blockage or ambiguity).
COM: complexity barrier (e.g., no MC/DC witness found within budget).
MIS: requirement misinterpretation (traceability mismatch).

4. Results

4.1. Coverage Metrics

Across the **32 experiments** (micro-averaged), agents achieved high line/branch coverage for ASIL A/B, but MC/DC independence remained unmet for the ASIL D experiment; boundary-value gaps persisted. Per-agent summaries are shown in Table 1.

4.2. Failure Patterns

Table 2 reports the failure patterns per ASIL-specific test agent, micro-averaged over all experiments. Each row summarises the agent–unit tasks attempted by that agent and shows the frequency of the observed error types. Specifically, **FP** (False Positive) denotes a case where a test fails although the reference oracle passes, **INC** (Inconsistency) refers to contradictory outputs across runs or artefacts, **COM** (Complexity Barrier) indicates that no MC/DC witness could be found within the given budget, and **MIS** (Misinterpretation) marks a mismatch between the generated test and the requirement. The numeric values represent absolute counts per category, while "Success (%)" indicates the proportion of tasks completed without any failure.

4.3. Autonomy Metrics

Table 3 summarises autonomy per agent as micro-averages over all experiments (agent–unit tasks). *First-Pass Success (FPS)* is the fraction of experiments in which the very first generation achieved the ASIL-specific coverage goal without any re-prompting. *Avg. Iterations* is the mean number of re-prompt cycles per experiment until that goal was met; experiments that did not meet the goal within the loop budget are counted with the budget value (≤ 3). *Tasks (n)* lists how many experiments each agent attempted. Higher FPS and lower iteration counts indicate greater autonomy.

Table 3
Autonomy per agent (micro-averages over experiments).

ASIL	Tasks (n)	First Pass Success (%)	Avg. Iterations ($\leq$3)
A	23	83	1.4
B	8	88	1.2
D	1	50	2.0

5. Discussion and LLAIS Perspective

Coverage results show that nominally high averages mask critical MC/DC and boundary gaps. GPT-4 Turbo achieved norm-compliant outputs for ASIL A/B, but MC/DC remained unsolved for ASIL D.

The baseline is controlled; reproducibility in the full artefact-release sense will be addressed in future work, planned as a separate publication. This aligns with the LLAIS focus on transparent and repeatable evaluation, but here we scope our contribution to methodology and baseline results.

From a LLAIS perspective, the multi-agent architecture illustrates how LLMs can be orchestrated for safety-critical tasks, and could be extended with *retrieval-augmented generation (RAG)* and knowledge graphs for traceability [7, 8].

6. Related Work

Test generation with LLMs. Recent studies report moderate coverage and edge-case gaps in LLM-generated tests, alongside industrial tooling for *improving* existing tests [9, 10, 11].
Agent orchestration in SE. Multi-agent frameworks (e.g., AutoGen/AutoAgents) enable specialised roles and coordination for complex SE tasks [7, 8].
MC/DC and structural coverage. Practical guidance for MC/DC (unique-cause) and its certification context is well established in avionics and relevant to ASIL D [5, 6].

Our contribution integrates these lines of research into a safety-critical context (ASPICE SWE.4, ISO 26262-6).

7. Ethics and Limitations

Risks include:

- **Coverage $\neq$ Compliance**: misleading evidence if metrics are misinterpreted.
- **Spurious evidence**: LLM outputs may be plausible but invalid.
- **Regulatory**: safety standards require human oversight; LLMs cannot replace assessors.

We recommend human-in-the-loop validation and transparency in reporting.

Threats to validity: limited test set (internal), simplified functions (external), definition of success (construct), and small sample size (conclusion).

8. Conclusion and Future Work

This paper presents the first controlled multi-agent GPT-4 Turbo baseline for SWE.4-compliant unit testing with ISO 26262-6 evidence.

Findings:

- ASIL A/B agents achieve norm-compliant coverage with few errors.
- ASIL D agent fails MC/DC independence.
- Coverage metrics overstate sufficiency; autonomy metrics reveal iteration needs.

Table 4
MC/DC matrix (simplified). Each condition independently affects decision outcome.

A	B	C	D	Outcome
0	0	0	0	0
1	0	0	0	1
0	1	0	0	1
0	0	1	0	1
0	0	0	1	1

Future work will: extend the dataset, test GPT-4o/GPT-5, add RAG/knowledge-graph orchestration, and integrate automated MC/DC gap detection.

A. Macro vs. Micro Averaging

In this paper, unless stated otherwise, we use *micro-averages*: each experiment (agent–unit task) contributes equally to the mean. This is different from *macro-averaging*, where each code unit is given equal weight regardless of the number of experiments performed on it. Formally, let E be the set of experiments and U the set of units:

- **Micro-average:** Micro $= \frac{1}{|E|} \sum_{e \in E} \mathrm{metric}(e)$.
- **Macro-average:** First compute per-unit means $c(u) = \frac{1}{|E_u|} \sum_{e \in E_u} \mathrm{metric}(e)$, then average over units: Macro $= \frac{1}{|U|} \sum_{u \in U} c(u)$.

Macro-averaging prevents units with many experiments from dominating the overall result, while micro-averaging reflects the distribution of all individual tasks. All agent-level tables in this paper report *micro-averages*.

B. MC/DC Example

We verify MC/DC independence using decision/condition matrices. Example (NASA case [5]):

References

[1] OpenAI, New models and developer products announced at devday, https://openai.com/index/new-models-and-developer-products-announced-at-devday/, 2023. Accessed: 2025-08-15.

[2] OpenAI, Gpt-4 technical report, arXiv preprint arXiv:2303.08774 (2023). URL: https://arxiv.org/abs/2303.08774. doi:10.48550/arXiv.2303.08774.

[3] International Organization for Standardization, Iso 26262-6:2018 road vehicles – functional safety – part 6: Product development at the software level, 2018. doi:10.3403/30344093.

[4] VDA QMC, Automotive spice process assessment model (pam) version 4.0, 2021. URL: https://www.automotivespice.com/fileadmin/software-download/ASPICE_PAM_40.pdf, verband der Automobilindustrie (VDA) Quality Management Center.

[5] J. J. Chilenski, An Investigation of Three Forms of the Modified Condition Decision Coverage (MCDC) Criterion, Technical Report NASA/TM-2001-210876, NASA Langley Research Center, 2001. URL: https://ntrs.nasa.gov/citations/20010085946.

[6] J. J. Chilenski, S. P. Miller, An Investigation of Three Forms of the Modified Condition/Decision Coverage (MC/DC) Criterion, Technical Report DOT/FAA/AR-01/18, Federal Aviation Administration, 2001.

[7] Q. Wu, G. Bansal, J. Zhang, et al., Autogen: Enabling next-gen llm applications via multi-agent conversation, arXiv preprint arXiv:2308.08155 (2023). URL: https://arxiv.org/abs/2308.08155.

[8] G. Chen, S. Dong, et al., Autoagents: A framework for automatic agent generation, arXiv preprint arXiv:2309.17288 (2023). URL: https://arxiv.org/abs/2309.17288.
[9] N. Alshahwan, J. Bader, et al., Automated unit test improvement using large language models at meta, arXiv preprint arXiv:2402.09171 (2024). URL: https://arxiv.org/abs/2402.09171.
[10] Y. Zhang, X. Li, et al., An empirical study of unit test generation with large language models, arXiv preprint arXiv:2406.18181 (2024). URL: https://arxiv.org/abs/2406.18181.
[11] M. Farias, et al., A system for automated unit test generation using large language models (agonetest), arXiv preprint arXiv:2408.07846 (2024). URL: https://arxiv.org/abs/2408.07846.